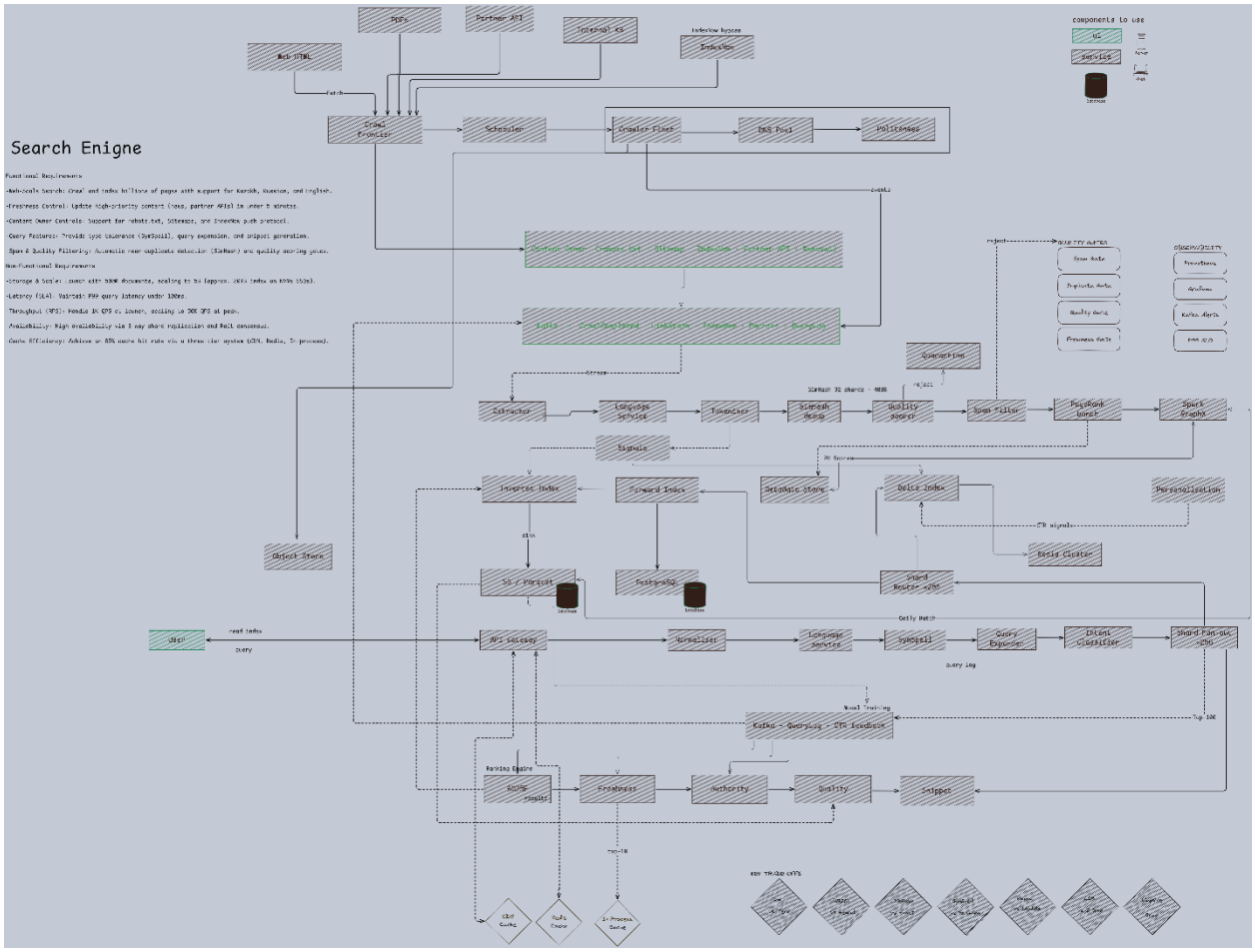
Full name: Abdykamat Adilet

ID: 24B030970



KZ Web-Scale Search Engine — System Design 500M → 5B documents | 1K → 50K QPS | P99 < 100ms | Kazakh · Russian · English

1. Problem & Scope

A national search engine must crawl billions of pages, serve sub-100ms queries with relevance, typo tolerance, and personalisation, while preventing spam and giving content owners indexing control. Corpus: 500M docs at launch, 5B at scale. Languages: Kazakh (agglutinative, custom tokeniser) + Russian + English. Freshness SLA: under 5 min for news, under 24h for general web, weekly for archives. Quality bar: filter duplicates, link farms, thin AI content.

2. Architecture: Six Subsystems (one-way flow)

Crawl → Process → Index → Serve. No query-time coupling back to crawl.

Crawler: distributed fleet, DNS pool, robots.txt cache, IndexNow listener. Processing: Kafka + Spark Structured Streaming, SimHash dedup, language detection, link graph builder. Indexing: inverted index, forward index, metadata

store, 256 shards, LSM delta merge. Ranking: BM25F × PageRank × freshness × quality. Query processor: SymSpell correction, synonym expansion, intent classifier, scatter-gather. Serving: CDN + Redis cache + API gateway, 85% cache hit rate.

3. Crawling Strategy

Priority queue scored on: PageRank estimate (0.35), freshness urgency (0.25), domain authority (0.20), user demand (0.15), depth from seed (0.05).Adaptive scheduling: news re-crawled every 5–15 min, blogs every 6–24h, PDFs every 7–30 days. Scheduler uses Redis sorted set with SETNX locks to prevent duplicate fetches across 200 stateless workers (4K fetch/s at scale).IndexNow push: content owners POST changed URLs, bypassing the queue — index lag under 2 minutes.

4. Processing Pipeline (Spark Streaming)

Each stage is idempotent, keyed on content hash: Extract → Language detect (FastText, <1ms) → Tokenise (KZ morphology / RU Mystem) → SimHash dedup (64-bit, Hamming ≤3, O(1) banding) → Quality score (reject <0.15) → Spam filter (link-farm + cloaking) → Link graph update → Signal extraction (CTR, anchor text, social shares). SimHash stores 5B fingerprints in 40GB across 32 in-memory shards. PageRank computed daily via Spark GraphX on 250B-edge graph; incremental 2-hop updates between runs.

5. Indexing

Inverted index: term → posting list (docID, TF, positions). Forward index: docID → title, snippet, quality, freshness. Metadata store: docID → PageRank, CTR, domain authority. 256 document-partitioned shards (hash docID mod 256). Each shard holds ~20M docs. All 256 queries run in parallel — 20ms budget per fan-out, 10ms to merge, 70ms headroom within 100ms SLA. Two-phase writes: new docs go to in-memory delta index first (holds 1M docs), merged into main index every 10 minutes via LSM — 10–20× faster than in-place B-tree updates on SSD.

6. Ranking Formula

score = BM25F^0.50 × authority^0.20 × freshness^0.15 × quality^0.15

BM25F field weights: title ×3.0, anchor text ×2.5, headings ×2.0, URL ×1.5, body ×1.0. Freshness: sigmoid decay — news threshold 1 day, informational threshold 365 days. Phase 2: LightGBM LTR model replaces hand-tuned formula once click data accumulates. Personalisation: language boost ×1.2, freshness weight ×2 for fresh-clicking users, domain familiarity ×1.1.

Intent classifier (logistic regression, 0.5ms): news → freshness weight 0.30 + Top Stories carousel; navigational → skip to 1 shard; entity → Knowledge Panel.

7. Query Processing (10 steps, total < 100ms)

Normalise (0.1ms) → Lang detect (0.5ms) → SymSpell correct (1ms, O(1) lookup) → Expand synonyms (1ms) → Intent classify (0.5ms) → Fan-out 256 shards (20ms) → Per-shard top-100 (15ms) → Global merge top-10 (10ms) → Snippet generate (5ms) → Cache write (1ms).

Roaring Bitmaps for faceted filters — narrows 20M candidates to 50K in under 2ms.

8. Caching (85% combined hit rate)

CDN edge: non-personalised queries, TTL 5min — 60–70% hit rate. Redis result cache: queries issued >10×/hour, TTL 60s (news) / 5min (general) — 25–35%. In-process LRU: parsed query objects, 256MB/node, TTL 10s — 10–15%.

At 50K QPS only 7,500 QPS reach the index. Probabilistic early recomputation prevents cache stampede.

9. Content Owner Controls

robots.txt / Sitemap: < 24h lag. IndexNow push: < 2min lag. Partner API (direct ingest): < 5min lag. Removal request (tombstone): < 10min. Crawl rate control via meta tag or HTTP header.

10. Key Trade-offs

Doc sharding vs term sharding: doc sharding chosen — simpler, each shard independent, hot-term problem avoided. BM25F vs neural day-1: BM25F <1ms, interpretable, competitive on short queries — neural added phase 2. SimHash vs exact hash: exact misses syndication (20 news sites, same article) — SimHash Hamming ≤3 catches it at O(1). SymSpell vs seq2seq: SymSpell O(1) vs seq2seq +50ms — wrong choice at 100ms SLA. Kappa vs Lambda: only PageRank is batch — full Lambda pipeline adds overhead for no benefit. LSM vs B-tree: LSM 10–20× faster sequential writes, absorbs bursts without blocking reads. Adaptive crawl: 80% low-change docs crawled weekly, 20% high-change drive SLA — saves 60% crawl infra cost.